Compte rendu TP N°1

Python - Partie 1

Module : Distributed Agentic System

Ahmed MILOUDI

8 octobre 2026

Objectif du TP

Ce travail pratique porte sur les conversions de types, les structures conditionnelles, les boucles, les fonctions et les principales structures de données Python. Les exercices sont réalisés dans un notebook Jupyter avec Visual Studio Code et l'environnement virtuel `.venv`.

Table des matières

1 Conversions et moyenne pondérée **3**
1.1 Principe . . . 3
1.2 Réponse . . . 3
1.3 Exemple . . . 3

2 Structures conditionnelles : équation du second degré **4**
2.1 Principe . . . 4
2.2 Réponse . . . 4
2.3 Exemples . . . 5

3 Boucles : contrôle d'un entier impair **5**
3.1 Réponse . . . 5

4 Fonctions, paramètre par défaut, map et zip **5**
4.1 Réponse . . . 5
4.2 Résultat . . . 6

5 Listes et calcul de la médiane **6**
5.1 Principe . . . 6
5.2 Réponse . . . 6
5.3 Exemple . . . 7

6 Dictionnaires, listes et tuples pour un système multi-agents **7**
6.1 Réponse 7

7 Conclusion **8**

1 Conversions et moyenne pondérée

1.1 Principe

La fonction `input()` renvoie toujours une chaîne de caractères. Il faut donc utiliser `float()` pour les notes et `int()` pour les coefficients. La moyenne pondérée est calculée avec la formule :

$$\text{moyenne} = \frac{\text{note}_{TP} \times \text{coefficient}_{TP} + \text{note}_{examen} \times \text{coefficient}_{examen}}{\text{coefficient}_{TP} + \text{coefficient}_{examen}} \tag{1}$$

1.2 Réponse

```python
def saisir_float(message):
    while True:
        saisie = input(message).strip().replace(",", ".")
        try:
            return float(saisie)
        except ValueError:
            print("Erreur : saisissez un nombre decimal.")

def saisir_entier(message):
    while True:
        saisie = input(message).strip()
        try:
            return int(saisie)
        except ValueError:
            print("Erreur : saisissez un nombre entier.")

identifiant = input("Identifiant de l'etudiant : ").strip()
note_projet = saisir_float("Note du projet pratique (TP) : ")
note_ecrit = saisir_float("Note de l'examen ecrit : ")
coeff_projet = saisir_entier("Coefficient du projet : ")
coeff_ecrit = saisir_entier("Coefficient de l'ecrit : ")

total_coefficients = coeff_projet + coeff_ecrit
if total_coefficients == 0:
    print("Erreur : la somme des coefficients ne peut pas etre nulle.")
else:
    moyenne = (
        note_projet * coeff_projet + note_ecrit * coeff_ecrit
    ) / total_coefficients
    print(
        "Pour l'etudiant " + identifiant
        + ", la moyenne finale du module est de "
        + str(round(moyenne, 2)) + " / 20."
    )
```

1.3 Exemple

Pour les valeurs `16.5`, `14`, `2` et `3`, le résultat obtenu est :

```
Pour l'etudiant CNE123456, la moyenne finale du module est de 15.0 /
20.
```

2 Structures conditionnelles : équation du second degré

2.1 Principe

Pour une équation $ax^2 + bx + c = 0$, on calcule le discriminant :

$$\Delta = b^2 - 4ac \tag{2}$$

— si $\Delta > 0$, l'équation possède deux solutions réelles ;

— si $\Delta = 0$, elle possède une solution réelle double ;

— si $\Delta < 0$, elle ne possède aucune solution réelle.

2.2 Réponse

```python
def saisir_float(message):
    while True:
        saisie = input(message).strip().replace(",", ".")
        try:
            return float(saisie)
        except ValueError:
            print("Erreur : saisissez un nombre reel.")

a = saisir_float("Coefficient a : ")
b = saisir_float("Coefficient b : ")
c = saisir_float("Coefficient c : ")

if a == 0:
    if b == 0:
        est_second_degre = False
        if c == 0:
            print("Equation indeterminee : une infinite de solutions.")
        else:
            print("Equation degeneree : aucune solution.")
    else:
        est_second_degre = False
        print("Equation du premier degre.")
else:
    est_second_degre = True

if est_second_degre:
    delta = b ** 2 - 4 * a * c
    print(f"Discriminant Delta = {delta}")
    if delta > 0:
        print("L'equation possede deux solutions reelles.")
```

```
    elif delta == 0:
        print("L'equation possede une solution reelle double.")
    else:
        print("L'equation ne possede aucune solution reelle.")
```

2.3 Exemples

a	b	c	Δ	Conclusion
1	-3	2	1	Deux solutions réelles
1	2	1	0	Une solution double
1	0	1	-4	Aucune solution réelle

3 Boucles : contrôle d'un entier impair

3.1 Réponse

```
while True:
    saisie = input(
        "Veuillez saisir un nombre entier impair : "
    ).strip()

    if "." in saisie or "," in saisie:
        print("Erreur : vous avez saisi un nombre decimal.")
        continue

    try:
        nombre = int(saisie)
    except ValueError:
        print("Erreur : veuillez saisir un entier.")
        continue

    if nombre <= 0:
        print("Erreur : veuillez saisir un entier impair positif.")
    elif nombre % 2 == 0:
        print(f"Erreur : {nombre} est un nombre pair.")
    else:
        break

print(f"Saisie acceptee ! Voici les nombres de 0 a {nombre} :")
for valeur in range(nombre + 1):
    print(valeur)
```

La boucle `while` contrôle la saisie jusqu'à obtenir un entier impair positif. La boucle `for` affiche ensuite les valeurs de 0 jusqu'au nombre choisi.

4 Fonctions, paramètre par défaut, map et zip

4.1 Réponse

```python
def ajouter_bonus(note, bonus=2):
    return note + bonus

x = 10
resultat_x = ajouter_bonus(x)
print(f"Resultat de ajouter_bonus(x) : {resultat_x}")
print(f"Valeur initiale de x : {x}")

notes = [12.0, 15.5]
nouvelles_notes = list(map(ajouter_bonus, notes))
print(f"Notes apres bonus : {nouvelles_notes}")

etudiants = ["Ali", "Sara"]
for etudiant, note in zip(etudiants, nouvelles_notes):
    print(f"{etudiant} : {note}/20")
```

4.2 Résultat

```
Résultat de ajouter_bonus(x) : 12
Valeur initiale de x : 10
Notes après bonus : [14.0, 17.5]
Ali : 14.0/20
Sara : 17.5/20
```

La valeur de `x` reste égale à 10, ce qui montre l'immuabilité des types numériques simples.

5 Listes et calcul de la médiane

5.1 Principe

Les notes sont saisies dans l'ordre croissant. Si le nombre de notes est impair, la médiane est la valeur centrale. S'il est pair, elle est la moyenne des deux valeurs centrales :

$$\text{médiane} = \frac{\text{valeur centrale } 1 + \text{valeur centrale } 2}{2} \tag{3}$$

5.2 Réponse

```python
while True:
    try:
        nombre_notes = int(
            input("Combien de notes voulez-vous saisir ? ").strip()
        )
        if nombre_notes <= 0:
            raise ValueError
        break
    except ValueError:
```

```
        print("Erreur : saisissez un nombre entier positif.")

notes = []
for i in range(nombre_notes):
    while True:
        try:
            note = float(
                input(f"Note {i + 1} : ").strip().replace(",", ".")
            )
        except ValueError:
            print("Erreur : saisissez une note numerique.")
            continue

        if notes and note < notes[-1]:
            print(
                f"Erreur : la note doit etre superieure ou egale "
                f"a {notes[-1]}."
            )
            continue
        break
    notes.append(note)

print(f"Liste des notes : {notes}")
if nombre_notes % 2 == 1:
    mediane = notes[nombre_notes // 2]
else:
    milieu = nombre_notes // 2
    mediane = (notes[milieu - 1] + notes[milieu]) / 2

print(f"La mediane est : {mediane}")
```

5.3 Exemple

Avec les notes `10, 12, 16, 20`, la médiane vaut :

$$\frac{12+16}{2} = 14 \tag{4}$$

6 Dictionnaires, listes et tuples pour un système multi-agents

6.1 Réponse

```
config_ia = {
    "agent_recherche": {
        "modele": "gpt-4",
        "temperature": 0.2,
        "outils": [
            ("WebSearch", "v2.1"),
```

```
            ("Calculatrice", "v1.0")
        ]
    }
}

config_ia["agent_recherche"]["temperature"] = 0.5
print(config_ia["agent_recherche"]["outils"][1][0])
```

La dernière instruction navigue dans le dictionnaire, puis dans la liste et enfin dans le premier élément du tuple. Elle affiche :

```
Calculatrice
```

7 Conclusion

Ce TP a permis de mettre en pratique les conversions de types, les conditions, les boucles, les fonctions et les structures de données Python. Ces notions constituent une base nécessaire pour organiser les données et le comportement des agents dans un système distribué agentique.